

Homework 3: Design the Evaluation Before Running It

Statistical Evaluation of AI · Spring 2027

Out / due	Tue Feb 2 / Tue Feb 9, 11:59pm
Points	100: Problem 1 (50), Problem 2 (30), Problem 3 (20)
Expected time	About 5 hours after completing Lab 3; contact course staff if setup/debugging alone exceeds 30 minutes
Reading	Mitchell et al., “ Model Cards for Model Reporting ”, Section 4, especially intended use, metrics, evaluation data, and caveats
AI policy	No AI for Problem 1; allowed in Problem 2 and required in Problem 3. Submit the AI-use record.
Working file	Complete all work in <code>hw03_starter.ipynb</code> ; the PDF is a read-only prompt copy.
Submit	Completed notebook PDF + completed <code>.ipynb</code> in one Gradescope submission.

Problem 1 - Plan the scikit-learn study (50 points; no AI)

In this problem, you will decide what the study should measure and how it should be run. You will make those decisions before seeing any model results.

Why no AI here? The exam will ask you to make and defend choices like these on your own. This problem gives you a chance to practice with feedback.

The setting

When someone reports a problem on scikit-learn’s GitHub page, a maintainer may add a label such as `module:metrics` or `module:tree`. These labels help route the issue to someone with the right expertise. They do not assign a particular person, and they do not determine whether the issue will be fixed.

The project is considering a tool that recommends one of eight module labels from the issue title. The current method uses TF-IDF features and multinomial logistic regression. The proposed method uses a transformer. In either case, a maintainer makes the final choice.

The eight labels are `module:cluster`, `module:ensemble`, `module:linear_model`, `module:metrics`, `module:model_selection`, `module:preprocessing`, `module:tree`, and `module:utils`.

You have a file called `github_issues/issues.csv`. It contains 120 real, closed scikit-learn issues created between 2018 and 2025: 15 from each of eight module labels. For each issue, the file gives the issue number, title, creation date, recorded module label, a pseudonymized reporter group, and a link to the original issue.

This is a deliberately balanced teaching dataset, not a random sample of all incoming issues. It leaves out issue bodies, comments, usernames, issues with more than one module label, and modules outside the selected eight.

a. **What does the recorded label mean? (10 points)**

(i) Let Y_i be the module label recorded for issue i . In 1-2 sentences, explain what this label tells us. Why is it not proof that the issue has one objectively correct module label? Give one reason the recorded label could have been different. (4 points)

(ii) Name one group of GitHub issues that `github_issues/issues.csv` directly describes. Then name one broader group of GitHub issues that the dataset does not necessarily represent. (3 points)

(iii) Imagine that an analyst combines two module labels after discovering that the new grouping makes the transformer look better. Why is this more than ordinary data cleaning? (3 points)

b. **Decide what would count as a better model (14 points)**

The study compares three approaches:

1. A **baseline** ignores the issue title and always predicts one module label: the most common label in the training data, with alphabetical order breaking a tie. This is the **no-information rule** for this study. The full 120-row dataset is balanced, but the baseline's accuracy on the final test set will depend on that test set's label mix.
2. The **simple title model** uses TF-IDF features and multinomial logistic regression.
3. The **transformer** is the more complex title model being considered.

The method names are already given; you do not need to copy them into the table. Instead, plan two checkpoints:

- **Checkpoint 1 - Do titles help at all?** Compare the simple title model with the baseline on the same test issues. Plan to report an uncertainty interval for the difference. This checkpoint passes if the entire interval is above zero.
- **Checkpoint 2 - Is the transformer worth it?** If the simple model helps, compare the transformer with the simple model on those same test issues. Choose the minimum additional improvement that would justify piloting the more complex model. This minimum is the **practical threshold**.

Choose one main performance measure:

- **Accuracy:** the percentage of GitHub issues assigned the recorded module label. This is the simplest option and gives every test issue equal weight. A module contributes more when it is more common in the chosen test set.
- **Average recall across modules:** compute recall separately for each of the eight module labels, then average the eight values. This is also called macro recall or balanced accuracy. Use it only if you can explain why equal weight for every module fits the intended use.

Complete the table in the notebook. State what the model predicts and how a maintainer would use it, which scikit-learn GitHub issues the conclusion should cover, your performance measure, and the rule for passing each checkpoint. Write both model differences in words; these are the two quantities the study is designed to estimate.

c. **Choose how to split the 120 GitHub issues (14 points)**

The dataset has 120 rows. Each row is one scikit-learn GitHub issue, and the model makes one module-label prediction for that issue. The `reporter_group` column identifies GitHub issues that came from the same pseudonymized reporter.

Use three sets of data:

- **Training set:** fit the models and any text preprocessing.
- **Validation set:** choose model settings and make any allowed design choices without using the test results.
- **Test set (final holdout):** compare the two finished models once, after all choices have been made.

How you assign GitHub issues to these sets depends on the question you want to answer. A time-based split can train on older issues and test on newer issues. A reporter-held-out split keeps each reporter group in only one set and tests performance on reporters the model has not seen. A random row split assigns individual GitHub issues at random. It estimates performance for another issue drawn from roughly the same mix of years and reporters as this dataset. It does not directly test performance on future issues or on entirely new reporters.

The starter notebook displays the number of issues from each module in each year. Use that EDA to make sure your proposed training, validation, and test periods are workable for all eight labels.

Complete the split table in words. You do not need to write code or actually divide the rows. Work through these decisions:

1. Finish this sentence: “Our final comparison is meant to tell us how the two models perform on _____.” Be specific. For example, you might care about later scikit-learn issues in the same eight modules or issues submitted by reporters not seen during training.
2. Decide whether GitHub issues from the same `reporter_group` must remain in the same set. Explain why or why not.
3. Give a concrete rule for assigning the 120 rows to training, validation, and test sets. You may use the creation dates, reporter groups, or both.
4. Explain why your rule fits the group of GitHub issues named in step 1. State whether it is mainly protecting against changes over time, repeated reporters across sets, or both.
5. In one sentence each, explain what a random row split and a reporter-held-out split would tell you instead.

Use the final row of the table to explain why no split of `github_issues/issues.csv` can establish that the model works for a different software project. The completed table is your full answer; no separate essay is required.

d. Put the plan together (12 points)

This is a written planning exercise. Do not write code, fit either model, or calculate results. Complete the design-brief table in the notebook using words and your answers above. The brief should contain:

- your research question and hypothesis;
- the separate jobs of the training, validation, and test data;
- an if-then rule that checks both steps: whether the simple title model beats the no-information rule, and whether the transformer clears your minimum improvement over the simple model; and
- one important conclusion that even a favorable result would not support.

Following the Model Cards reading, also name the tool's intended use and one use that should be considered out of scope. After the table, write 2-3 sentences that summarize the plan and cite the relevant part of Section 4 once.

Problem 2 - Use EDA and run a complaint-routing baseline (30 points)

Now apply the statistical-design workflow to the CFPB complaint-routing example from Lab 3. You will use EDA, write a small amount of code, and run the supplied model scaffold.

The question for this problem is fixed:

Using public complaint narratives from 2022-2024, how well can a simple text classifier route public narratives received in 2025 among the five recorded product categories?

The file contains 400 public complaint narratives, with 20 complaints from each product-by-year combination. It also includes a `similarity_group` for repeated normalized narratives. You will compare a no-information classifier with TF-IDF multinomial logistic regression. You will not fit a transformer here; the goal is to build and check the baseline workflow that a later model would have to beat.

Here is the weak initial workflow. The analyst randomly split rows from all four years, did not check whether repeated narratives crossed sets, chose which words to retain using all available narratives, and kept checking test results while changing settings. In the parts below, replace those four choices one at a time and use code or output to document the resulting design.

a. Inspect the data before modeling (4 points)

Run the supplied profiling code, which prints:

- the number of rows for every product and year;
- the number of `similarity_group` values containing more than one row; and
- the 10th, 50th, and 90th percentiles of `n_words`.

Keep the output visible. In 2-3 sentences, explain what the balanced sampling means for this exercise and why these public narratives do not represent all CFPB complaints. Add one assertion that verifies every product-year cell has 20 rows.

b. Build and check the time split (6 points)

Use the supplied scaffold to create:

- training data from 2022-2023;
- validation data from 2024; and
- test data from 2025.

Add two assertions of your own: one checking the expected number of rows in each set, and one checking that all five products appear in each set. Run the provided similarity-group check and preserve its output. Do not fit a model or look at test performance yet.

In one sentence, explain why the time split matches the stated 2025 question better than a random row split.

c. Fit preprocessing inside the model pipeline (6 points)

Run the supplied validation loop for `min_df` (**minimum document frequency**) values 1, 2, and 5. Each candidate must fit `TfidfVectorizer` only on the training narratives, followed by multinomial logistic regression. Keep the validation-results table visible and record which `min_df` it selects.

Add one line of code that verifies the selected value came from the candidate list. Then explain why fitting TF-IDF separately inside each candidate pipeline prevents the validation and test narratives from influencing which words are kept.

d. Open the test set once (5 points)

Only after parts (a)-(c) are complete, change `OPEN_TEST` from `False` to `True`. The supplied code will refit the selected pipeline on the combined training and validation rows, fit a no-information classifier, and evaluate both on the 2025 test rows. It also reports a paired bootstrap interval for the accuracy difference.

Preserve the output. State the two test accuracies, the estimated improvement, and its interval. Does the interval clear zero? In this complaint-routing example, that is the same kind of Checkpoint 1 as in Problem 1: does the simple text model beat the no-information rule?

e. Document the design changes (4 points)

Complete the four-row design table in the notebook. For each row, name the problem in the initial workflow, state the design decision, and point to the supporting code or output: time split, similarity-group check, TF-IDF fitting, and one-time test evaluation.

Finish with two sentences: one limitation reduced by the code and one that the code cannot solve. The remaining limitation must discuss selection into the public-narrative dataset.

f. AI stretch (required): simulate duplicate contamination (5 points)

Before using AI, predict which split will look better and why. Then use AI to help write a small, reproducible simulation with repeated groups and a group-specific fingerprint. Compare the same simple classifier under:

1. a random row split that can place rows from one group in both training and test data; and
2. a group-held-out split with no group shared across the boundary.

Use a fixed seed, preserve the final code and output, and verify the group overlap for both splits. Report both accuracies and explain whether the result matched your prediction. Finally, connect the simulation to `similarity_group` in the complaint data and name one way the simulation is less realistic than actual complaint narratives.

Record the initial AI prompt and your checks in the AI-use table. Full credit does not require the random-row result to be larger; a surprising or failed result can earn full credit when the code and diagnosis are accurate.

Problem 3 - See what a detailed prompt changes (20 points)

Problem 2 gave you a worked complaint-routing example. Now return to the scikit-learn study you designed in Problem 1. You will ask the same AI tool and model for two analysis plans. Start a fresh conversation for each request so the first answer cannot influence the second. Give both conversations the same `issues.csv` file, or the same description of the data and its columns.

- **Original request:** “Use the supplied scikit-learn issues to determine whether the transformer is better than logistic regression for issue triage.” Ask the assistant to propose an analysis plan.
- **Detailed request:** Give the assistant your completed design brief from Problem 1 and ask for an analysis plan that could be carried out as written.

You will receive only one answer to each prompt. That is enough to compare the two answers, but not enough to prove that every difference was caused by the wording of the prompt. AI systems can give different answers even when asked the same question twice.

a. Compare the two answers (8 points)

Complete the five-row table in the notebook. Compare what each answer says about:

- the target population and outcome;
- the unit of analysis and data split;
- the comparison ladder: no-information rule, simple model, and transformer;
- uncertainty and the size of an improvement that would matter; and
- the conclusion and decision the evidence could support.

For each row, quote or point to specific wording in the response. If something important is missing, say so directly.

b. What improved, and what did not? (5 points)

Describe one meaningful improvement in the answer to the detailed request. Then describe one error or omission that remains. Use 1-2 sentences for each and point to evidence in the responses.

Add one sentence explaining why two answers are not enough to separate the effect of the prompt from ordinary variation in the model's responses.

c. Make the final call yourself (5 points)

Write the corrections an analyst would need before carrying out either plan. Do not ask the AI for a third rewrite. End by naming one study-design decision that remained your responsibility, even if the assistant happened to recommend the same choice.

Required AI-use record (2 points)

Complete the AI-use table in the notebook. Record the initial prompt for each use, but not the full conversation. Paste the two complete first responses in **Appendix A: Raw AI evidence**. Do not rely on an external link.